

Supplement to Lectures on Hadoop, Distributed File Systems and MapReduce

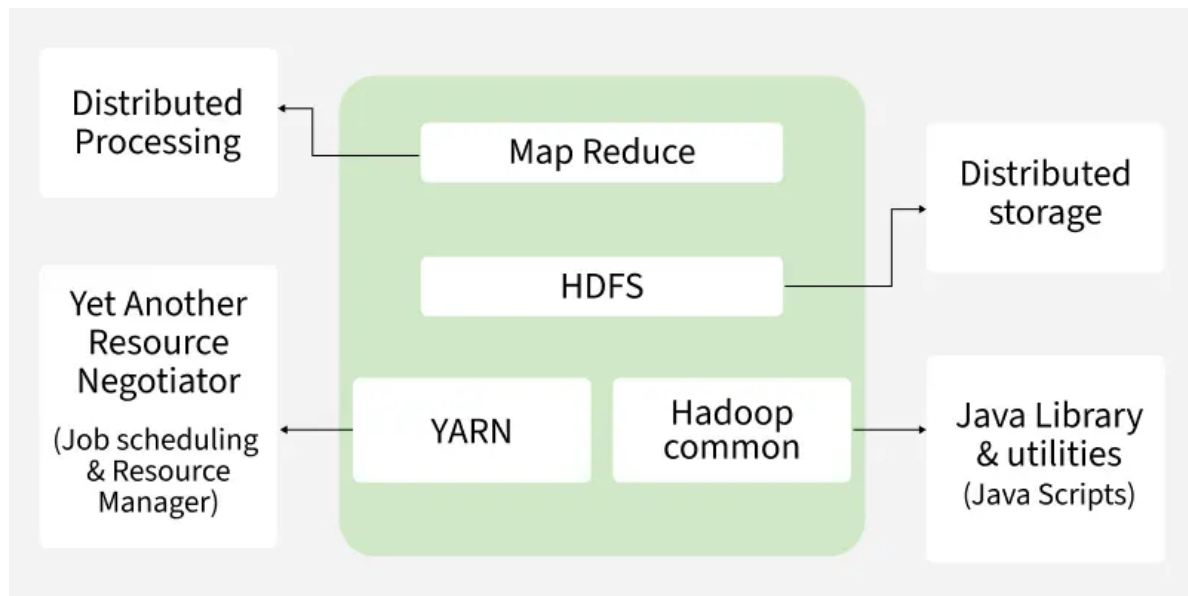
Hadoop - Architecture

Hadoop is an open-source Java framework that stores and processes massive data using clusters of inexpensive (commodity) hardware. Based on Google's MapReduce programming model, it enables distributed, parallel processing. Big companies like Facebook, Yahoo, Netflix, and eBay use Hadoop to efficiently handle large-scale data.

Components of Hadoop Architecture

Hadoop Architecture Mainly consists of 4 components:

- HDFS (Hadoop Distributed File System)
- YARN (Yet Another Resource Negotiator)
- MapReduce
- Common Utilities or Hadoop Common



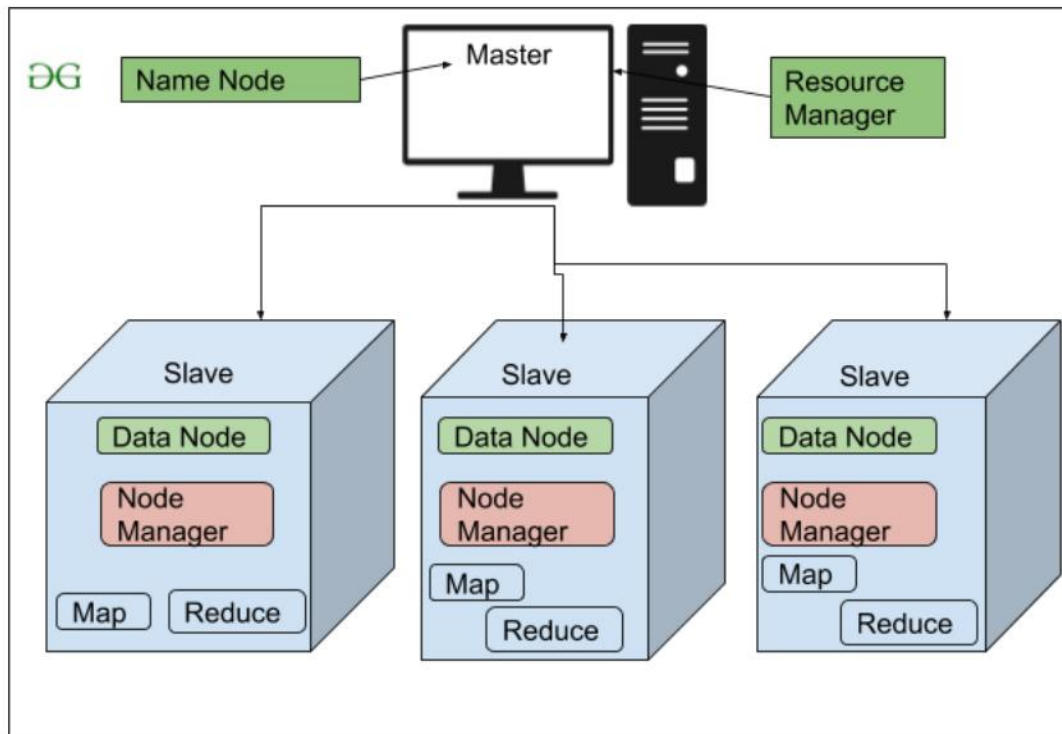
1. HDFS

HDFS (Hadoop Distributed File System) is Hadoop's primary storage system, built for high-throughput access to large datasets. It runs on inexpensive commodity hardware and stores data in large blocks to optimize performance. HDFS ensures fault tolerance and high availability across the cluster.

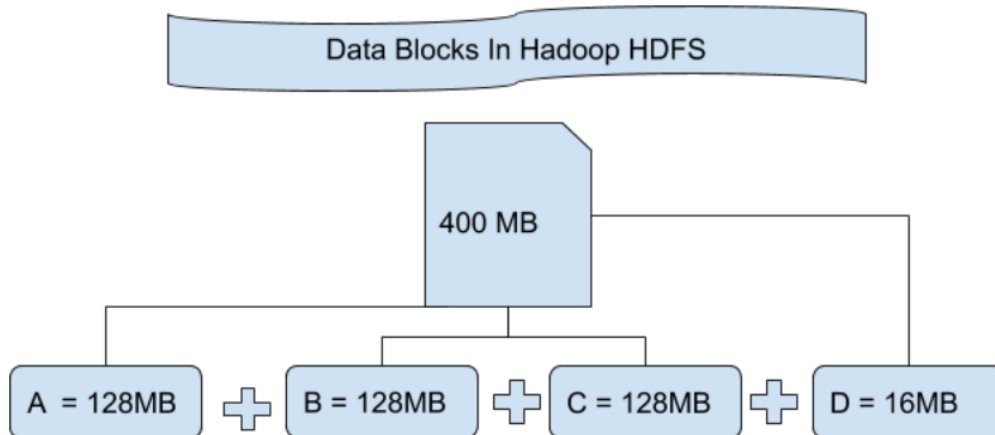
HDFS Architecture Components:

- **NameNode (Master Node):** The master node in HDFS that stores metadata (not actual data), manages file operations, and directs clients to the nearest DataNode for efficient access.
- **DataNode (Slave Node):** Stores actual data blocks, serves read/write requests, and reports to NameNode. Supports replication (default 3) for fault tolerance and scales storage and performance as more nodes are added.

High-Level Architecture of Hadoop



File Block in HDFS: In HDFS, data is always stored in the form of blocks. By default, each block is 128MB in size, although this value can be manually configured depending on the use case (commonly increased to 256MB or more in modern systems).



Suppose you upload a 400 MB file to HDFS. Hadoop will divide this file into blocks as follows:

$$128MB + 128MB + 128MB + 16MB = 400MB$$

This creates four blocks: three of 128 MB and one of 16 MB. Hadoop splits files purely by size, not content, so a single record can span across two blocks.

1. Comparison with Traditional File Systems

- Traditional file systems use small blocks (e.g., 4KB), while HDFS uses large blocks (128MB or more).
- Larger blocks in HDFS reduce metadata and I/O overhead, improving scalability and efficiency for big data processing.

2. Replication In HDFS:

HDFS replication ensures data availability and fault tolerance by storing multiple copies of each block.

- Default Replication Factor: 3
- If a file is split into 4 blocks, with a replication factor of 3: $4 \text{ blocks} \times 3 \text{ replicas} = 12 \text{ total blocks}$

Designed for commodity hardware, where failures are common, replication prevents data loss. While it increases storage usage, reliability is prioritized over space efficiency.

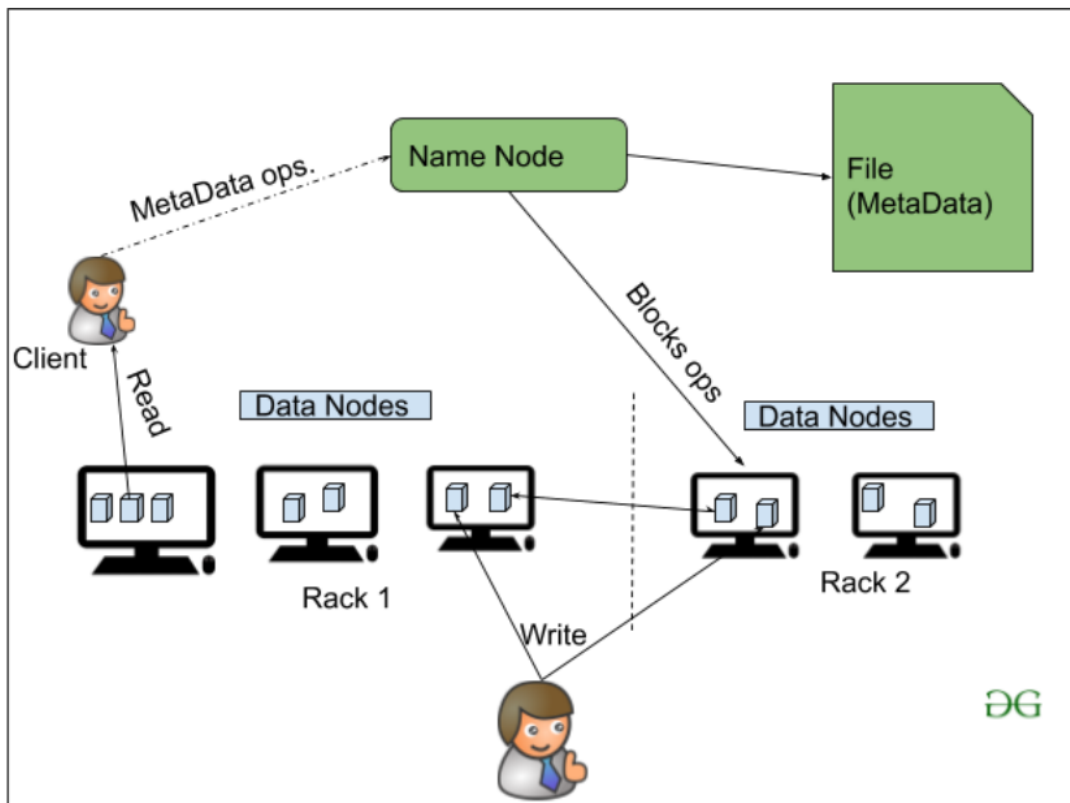
3. Rack Awareness:

A rack is a group of machines (typically 30–40) in a Hadoop cluster. Large clusters have many racks. Rack Awareness helps NameNode to:

- Choose the nearest DataNode for faster read/write operations.
- Reduce network traffic by minimizing inter-rack data transfer.

This improves overall performance and data access efficiency.

HDFS Architecture



2. YARN (Yet Another Resource Negotiator)

YARN is a resource management layer in the Hadoop ecosystem. It allows multiple data processing engines, such as MapReduce, Spark, and others, to run and share cluster resources efficiently.

It handles two core responsibilities:

- **Job Scheduling:** Splits large tasks into smaller jobs, assigns them to nodes, and manages priorities, dependencies, and execution.
- **Resource Management:** Allocates and monitors cluster resources (CPU, memory, etc.) needed for job execution.

Components of Yarn:

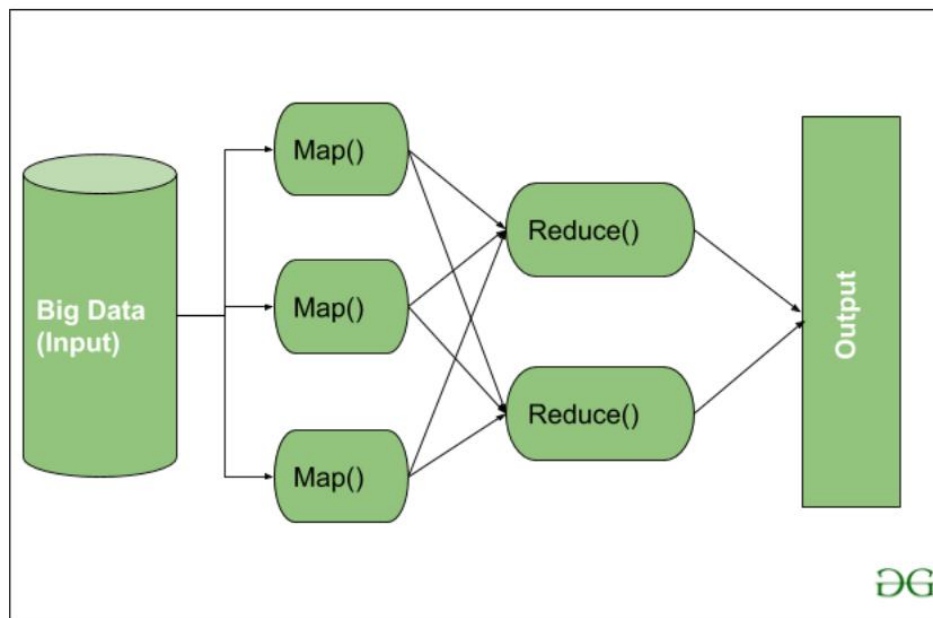
- **ResourceManager:** Master node that manages global resource allocation.
- **NodeManager:** Slave node that monitors resources on individual nodes.
- **ApplicationMaster:** Manages the lifecycle of each individual application/job.

Key Features of YARN:

- **Multi-Tenancy:** Supports multiple users and applications.
- **Scalability:** Efficiently scales to handle thousands of nodes and jobs.
- **Better Cluster Utilization:** Maximizes resource usage across the cluster.
- **Compatibility:** Works with MapReduce and other processing models like Spark.

3. MapReduce

MapReduce is a data processing model in Hadoop that runs on YARN. It enables fast, distributed, and parallel processing by dividing tasks into two phases, Map and Reduce, making it efficient for handling large-scale data.



MapReduce Workflow: workflow begins when input data is split into key-value pairs by Map() function. These are then grouped by key and processed by Reduce() function for tasks like sorting or aggregation. The final output is written to HDFS.

***Note:** Core aggregation often happens in the Reducer, but preprocessing and filtering are ideally done in the Mapper to optimize performance.*

Map Task Components:

- **RecordReader:** reads input data and converts it into key-value pairs, with keys as location info and values as actual data.
- **Mapper:** processes each pair and outputs zero or more new key-value pairs.
- **Combiner (optional):** acts as a mini-reducer to group Mapper output and reduce data transfer before shuffling.
- **Partitioner:** assigns key-value pairs to Reducers using `key.hashCode() % numberOfReducers`.

Reduce Task Components:

- **Shuffle and Sort:** transfers intermediate key-value pairs from Mappers to Reducers and sorts them by key. Shuffling begins as soon as some Mappers finish.
- **Reducer:** processes grouped key-value pairs, performing tasks like aggregation or filtering based on logic.
- **OutputFormat:** writes final results to HDFS using a `RecordWriter`, typically storing each record as a key-value line.

4. Hadoop Common (Common Utilities)

Hadoop Common, also known as Common Utilities, includes core Java libraries and scripts required by all components in a Hadoop ecosystem such as HDFS, YARN and MapReduce.

These libraries offer core functionalities such as:

- File system and I/O operations
- Configuration and logging
- Security and authentication
- Network communication

Hadoop Common provides shared libraries and utilities that help all Hadoop components work together. It handles hardware failures automatically and includes tools like Hadoop Archive, native library support, and RPC mechanisms.

Enhancements to Distributed File Systems

1. Comparison of DFS Architectures

Distributed File Systems (DFS) such as HDFS, GFS, Ceph, and Amazon S3 provide scalable storage for large-scale data processing.

HDFS and GFS are optimized for batch processing with block-level storage and replication. Ceph supports object, block, and file storage with high configurability, while Amazon S3 offers object-based storage accessible via APIs, ideal for cloud-native applications.

Feature	HDFS (Hadoop DFS)	GFS (Google File System)	Ceph	Amazon S3
Consistency	Strong consistency via single NameNode, but not POSIX-compliant. Clients write to a single replica, and others are updated.	Relaxed consistency; uses append-only writes; applications must tolerate stale reads.	Strong consistency with support for POSIX semantics; provides atomic writes.	Eventual consistency (now offers strong consistency in many regions as of 2020), especially for overwrite/delete.
Availability	High, but NameNode is a single point of failure unless configured with HA (High Availability).	The master node can be a bottleneck; it supports fault recovery.	Very high—fully distributed with no single point of failure; uses the CRUSH algorithm for replication.	Very high—managed by AWS across multiple availability zones with auto-replication.
Scalability	Scales well with large files and sequential workloads; limited by NameNode's memory (stores	Designed to scale for Google's internal workloads; metadata is handled by a single master,	Excellent scalability—true distributed metadata, dynamic scaling, no master bottleneck.	Virtually unlimited scalability; abstracts infrastructure from the user.

Feature	HDFS (Hadoop DFS)	GFS (Google File System)	Ceph	Amazon S3
	metadata in RAM).	limiting scalability.		
Performance	Optimized for high throughput and batch processing; suitable for large files and sequential access.	High throughput for append and sequential reads; less optimized for random access.	Good for both sequential and random access; suitable for block, object, and file workloads.	Very good for large object access; may show latency on small, frequent reads/writes. Not suitable for low-latency workloads.

2. Fault Tolerance Techniques

Fault tolerance in DFS ensures high availability and data durability even during hardware or network failures. HDFS achieves this through data replication (typically 3 copies). RAID configurations protect against disk-level failures within nodes. Erasure coding is a space-efficient alternative that splits and encodes data to reconstruct lost pieces.

3. Block Placement Strategies

Block placement is crucial for optimizing performance and ensuring fault tolerance. HDFS places blocks using a rack-aware policy to spread replicas across multiple racks. This enhances resilience to rack-level failures and supports efficient read performance through data locality. Understanding placement strategies illustrates how distributed storage is optimized for fault isolation and speed.

4. Security in DFS

Security features in DFS include authentication, authorization, and encryption. HDFS integrates with Kerberos for user authentication and supports POSIX permissions and ACLs for access control. Newer frameworks also implement encryption at rest and during transit, and integrate with tools like Apache Ranger for centralized policy management. Students should understand how these measures protect sensitive data in a shared infrastructure.

5. DFS in Modern Ecosystems

Modern DFS extensions such as Apache Ozone, Delta Lake, and Apache Iceberg provide transactional support, schema evolution, and streaming read/write capabilities across distributed storage. These systems bridge the gap between data lakes and data warehouses, enabling ACID compliance and time travel queries.

Enhancements MapReduce Programming Model

1. Chaining of MapReduce Jobs

In practice, many data processing tasks require multiple MapReduce jobs. For instance, computing TF-IDF scores from documents requires word count (Job 1) and normalization (Job 2).

These jobs are chained using tools such as Apache Oozie or manually managed using intermediate files.

Teaching this demonstrates how complex analytics are built from simpler stages.

2. Performance Bottlenecks and Optimizations

MapReduce jobs can suffer from performance issues due to straggler tasks, large shuffle sizes, and skewed data.

Speculative execution helps mitigate long-running tasks. Using combiners reduces data transferred during shuffle. Choosing between map-side or reduce-side joins affects efficiency.

Students should understand these trade-offs for optimizing large-scale processing jobs.

3. Beyond Word Count: Advanced Use Cases

MapReduce is used in many applications beyond word counting. Examples include creating inverted indexes for search engines, processing graph structures (e.g., PageRank), and sessionizing log data. These examples demonstrate MapReduce's versatility in real-world data engineering tasks.

4. Debugging and Monitoring Tools

Hadoop provides web UIs, such as the YARN Resource Manager and Job History Server, for tracking job progress and troubleshooting.

Task logs help diagnose failures and performance issues. Metrics like shuffle size and execution time inform tuning decisions.

Introducing these tools gives students hands-on strategies to debug and monitor distributed computations.

5. Limitations of MapReduce

Despite its strengths, MapReduce has limitations in iterative and real-time workloads. Each step writes to disk, which slows down ML and graph algorithms.

This led to the emergence of Apache Spark (in-memory computation), Flink (streaming), and other frameworks. Students should understand where MapReduce fits—and where it doesn't.

6. Transition to Modern Frameworks

Apache Spark surpasses MapReduce by offering in-memory processing, DAG execution, and rich APIs (RDD, DataFrame, SQL).

Its lazy evaluation model and support for batch, streaming, and ML workloads make it a modern replacement.

Highlighting this evolution gives students context on the trajectory of big data tools.